DOCUMENT 18 — PERFORMANCE, SCALABILITY & OPTIMIZATION SPECIFICATION

1. Dokümanın Amacı

Bu dokümanın amacı Smart Chef & Pantry AI uygulamasının Flutter mobil uygulaması, Flask backend'i ve PostgreSQL veritabanında performans gereksinimlerini, optimizasyon kurallarını ve gelecekteki ölçeklenebilirlik yaklaşımını tanımlamaktır.

Bu doküman özellikle aşağıdaki konuları kapsar:

- Flutter uygulama performansı
- API response süreleri
- Database query performansı
- Recipe recommendation performansı
- Ingredient search performansı
- Pagination
- Caching
- Network optimizasyonu
- Image optimizasyonu
- Backend scalability
- PostgreSQL indexing
- Concurrent request yönetimi
- Resource limits
- Performance monitoring
- Load testing
- MVP performans hedefleri
- Gelecekteki ölçeklenebilirlik yaklaşımı

Temel prensip:

Önce doğru mimari
↓
Sonra ölçüm
↓
Sonra darboğaz tespiti
↓
Sonra optimizasyon

Gereksiz erken optimizasyon yapılmayacaktır.

---

2. Performance Hedefleri

MVP için temel performans hedefleri:

İşlem	Hedef
Ingredient listesi	≤ 500 ms
Ingredient search	≤ 500 ms
Recipe detail	≤ 500 ms
Favorites listesi	≤ 500 ms
Cart listesi	≤ 500 ms
Recipe recommendation	≤ 1.5 sn
Basit API response	≤ 500 ms
Flutter ekran geçişi	Akıcı
Image loading	Progressive / cached
Cold start	Makul mobil uygulama açılış süresi

Bu değerler kesin SLA değildir.

Gerçek production verileri elde edildikten sonra:

p50
p95
p99

metrikleri üzerinden değerlendirme yapılacaktır.

---

3. Performance Ölçüm Yaklaşımı

Bir endpoint'in hızlı veya yavaş olduğuna yalnızca kullanıcı hissine göre karar verilmeyecektir.

Ölçülecek temel değerler:

Request count
Response time
Database query time
External service time
CPU usage
Memory usage
Error rate
Cache hit rate

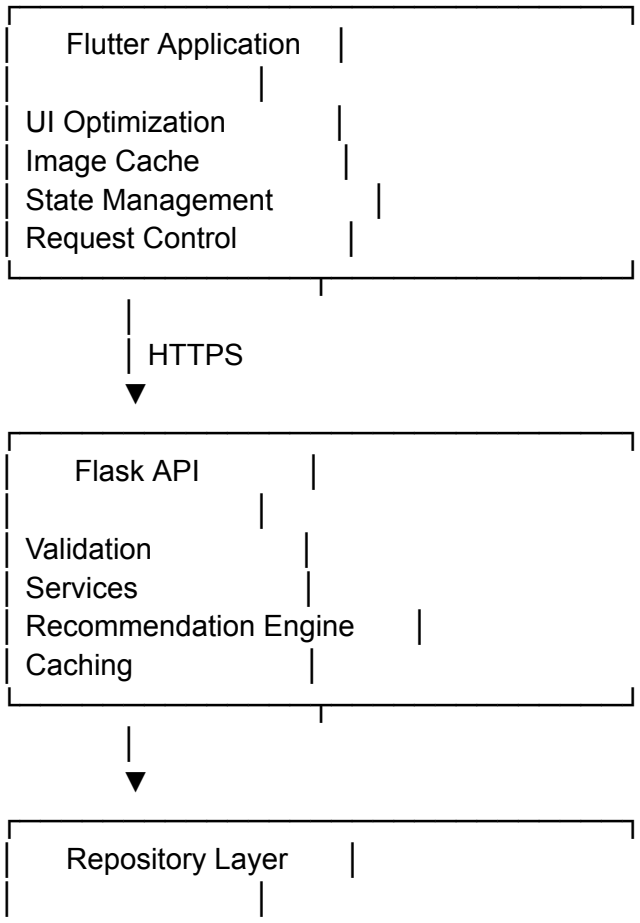
Özellikle API için:

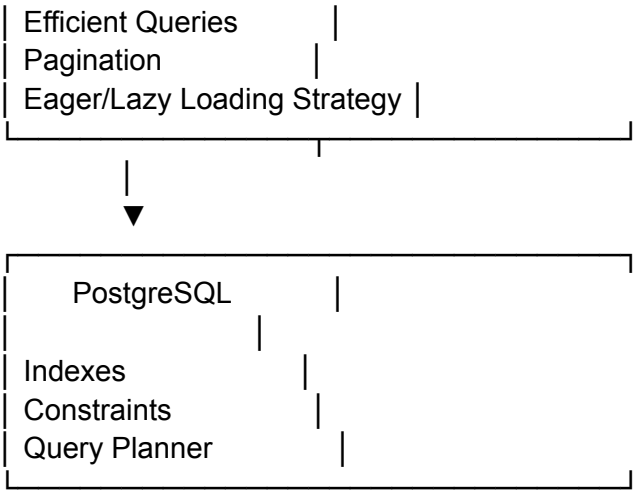
p50 → Ortalama kullanıcı deneyimine yakın
p95 → Yavaş kullanıcı isteklerini gösterir
p99 → Uç performans problemlerini gösterir

değerleri izlenmelidir.

---

4. Performance Architecture





---

5. Flutter Performance

Flutter tarafında performans için temel hedef:

UI thread gereksiz iş yükünden korunmalıdır.

Özellikle ağır işlemler UI içerisinde çalıştırılmamalıdır.

Yanlış:

UI
↓
1000 recipe üzerinde ağır matching
↓
UI freeze

Doğru:

Flutter
↓
API Request
↓
Backend Recommendation Engine
↓
JSON Response
↓
Flutter Rendering

Recommendation işlemi backend'de yapılacaktır.

---

6. Flutter State Management Performance

Riverpod kullanılırken gereksiz rebuild'lerden kaçınılmalıdır.

Örneğin bir cart item değiştiğinde:

Tüm uygulama

yerine yalnızca ilgili:

Cart Item
Cart Summary

widgetlarının güncellenmesi tercih edilmelidir.

Kullanılabilecek yaklaşım:

Provider
↓
State
↓
Selective Consumer
↓
Minimal rebuild

---

7. Widget Optimization

Flutter UI'da:

- Gereksiz `setState` kullanımından kaçınılmalı.
- Büyük listelerde `ListView.builder` tercih edilmeli.
- Büyük widget tree'leri gereksiz yere rebuild edilmemeli.
- Sabit widgetlar mümkün olduğunda `const` kullanılmalı.
- Ağır hesaplamalar build method içinde yapılmamalı.

Yanlış:

```
Widget build(BuildContext context) {
  final result = heavyCalculation();
  ...
}
```

Doğru yaklaşım:

Calculation
↓
Provider / Service
↓
State
↓
UI

---

8. Recipe List Performance

Recipe sonuçlarında çok sayıda tarif bulunabilir.

Bu nedenle:

ListView.builder

gibi lazy rendering kullanılmalıdır.

Bütün tariflerin aynı anda widget olarak oluşturulmasından kaçınılmalıdır.

Örnek:

1000 Recipes
↓
Only visible items rendered

---

9. Image Performance

Recipe görselleri uygulamanın performansını ciddi şekilde etkileyebilir.

Image optimizasyonunda:

Remote Image

↓
Cache
↓
Resize
↓
Display

yaklaşımı kullanılmalıdır.

Flutter tarafında `cached_network_image` kullanılabilir.

---

10. Image Size

Backend'den gereğinden büyük görseller gönderilmemelidir.

Örneğin:

Original:
4000 × 3000

mobil kart:

300 × 200

ise 4000×3000 görselin her seferinde indirilmesi gereksiz network ve memory tüketimine neden olur.

İleride image processing/CDN kullanılması değerlendirilebilir.

---

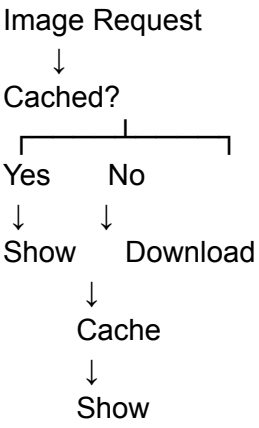
11. Image Placeholder

Görsel yüklenirken boş alan bırakmak yerine:

Loading Placeholder

kullanılabilir.

Akış:



---

12. Network Optimization

Flutter API isteklerinde:

Dio

kullanılacaktır.

Network performansı için:

- Gereksiz requestlerden kaçınılmalı.
- Aynı veri tekrar tekrar istenmemeli.
- Pagination kullanılmalı.
- Timeout tanımlanmalı.
- Request cancellation değerlendirilmeli.
- Response payload gereksiz büyütülmemeli.

---

13. Request Deduplication

Örneğin kullanıcı Search ekranında hızlı şekilde aynı sorguyu gönderirse:

domates
domates
domates
domates

için dört ayrı request göndermek gereksiz olabilir.

UI tarafında:

Debounce

kullanılabilir.

Örneğin kullanıcı:

d
do
dom
doma
domat
domates

yazarken her karakter için request göndermek yerine kısa bir bekleme uygulanabilir.

---

14. Search Debouncing

Ingredient search için:

User typing
↓
300–500 ms wait
↓
No new input?
↓
API request

yaklaşımı kullanılabilir.

Bu sayede:

API Requests ↓
Database Queries ↓
Network Usage ↓

olur.

Kesin debounce süresi gerçek kullanım testleriyle belirlenebilir.

---

15. API Payload Optimization

API response yalnızca gerekli alanları içermelidir.

Recipe recommendation response:

```
{
  "id": "...",
  "title": "...",
  "image_url": "...",
  "match_percentage": 80,
  "matched_ingredients": [],
  "missing_ingredients": []
}
```

şeklinde olabilir.

Liste ekranında gerekli olmayan:

long instructions
large descriptions

gibi alanların gönderilmesi gereksiz payload büyüklüğü oluşturur.

---

16. List Response ve Detail Response Ayrımı

Recipe list endpointi:

GET /api/recipes/recommend

özet bilgi döndürmelidir.

Recipe detail:

GET /api/recipes/{recipe_id}

detaylı bilgi döndürmelidir.

Örneğin:

List:
title
image
match percentage

Detail:
title
description
image
instructions
ingredients
source
youtube

Bu ayrım network kullanımını azaltır.

---

17. Pagination

Pagination performans için zorunlu tasarım prensiplerinden biridir.

Ingredient:

GET /api/ingredients?page=1&limit=20

gibi kullanılabilir.

Future recipe list endpointleri de pagination desteklemelidir.

Response:

```
{
  "success": true,
  "data": {
    "items": [],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 100
    }
  }
}
```

---

18. Pagination Limitleri

Client'ın:

limit=1000000

göndermesine izin verilmemelidir.

Backend:

MAX_PAGE_SIZE

uygulamalıdır.

Örneğin:

default = 20
maximum = 100

gibi değerler kullanılabilir.

Kesin değerler production kullanımına göre ayarlanmalıdır.

---

19. PostgreSQL Index Strategy

Database performance için indexler doğru kolonlarda kullanılmalıdır.

Önemli indexler:

ingredients.normalized_name
ingredients.category

recipe_ingredients.recipe_id
recipe_ingredients.ingredient_id

favorites.user_id
favorites.recipe_id

cart_items.user_id
cart_items.ingredient_id

Özellikle sık kullanılan query patternleri index tasarımının temelini oluşturmalıdır.

---

20. Index Overuse

Her kolona index eklemek doğru değildir.

Index:

Read performance ↑
Write overhead ↑
Storage ↑

oluşturabilir.

Bu nedenle yalnızca:

frequently queried
filtered
joined
sorted
unique

alanlarda gerekli indexler kullanılmalıdır.

---

21. Ingredient Search Optimization

Ingredient search:

normalized_name

alanı üzerinden yapılmalıdır.

Örneğin:

Tomato
tomato
 TOMATO

gibi değerlerin normalization sonrasında aynı şekilde ele alınması sağlanabilir.

Database query:

normalized_name

üzerindeki indexten yararlanmalıdır.

---

22. Recipe Recommendation Performance

Recommendation MVP'de:

User Ingredients
↓
Recipe Ingredients
↓
Match
↓
Score
↓
Sort

şeklinde çalışır.

Buradaki performans için bütün recipe verilerinin gereksiz şekilde Python memory'sine alınmasından kaçınılmalıdır.

Özellikle recipe sayısı büyüdüğünde:

Database filtering
+
Efficient joins
+
Indexes

kullanılmalıdır.

---

23. Recommendation Complexity

MVP'de basit yaklaşım:

Matched Required Ingredients
---------- × 100
Total Required Ingredients

şeklindedir.

Recipe sayısı:

R

ve recipe ingredient sayısı:

I

olduğunda naif yaklaşım gereksiz şekilde:

R × I

kontrolü oluşturabilir.

Veri büyüdüğünde database query optimizasyonu veya önceden hesaplanmış yardımcı yapılar değerlendirilebilir.

---

24. Recommendation Query Optimization

Örneğin kullanıcı:

tomato
onion
potato

seçtiğinde sistem bütün tarifleri ayrıntılı şekilde yüklemek yerine ilgili ingredient ilişkilerini database üzerinden filtreleyebilir.

Conceptual flow:

Selected Ingredient IDs
↓
recipe_ingredients
↓

Candidate Recipes
↓
Required Ingredient Count
↓
Matched Count
↓
Match %
↓
Ranking

Bu yaklaşım recipe sayısı büyüdüğünde daha ölçeklenebilir olacaktır.

---

25. Recommendation Caching

Aynı ingredient kombinasyonları sık tekrar ediliyorsa recommendation sonucu cache edilebilir.

Örneğin:

[ingredient-A, ingredient-B, ingredient-C]

normalize edilerek:

A-B-C

cache key oluşturulabilir.

Örnek:

recommendation:A-B-C

Ancak MVP'nin ilk sürümünde cache zorunlu değildir.

Öncelik:

Correctness
+
Database optimization
+
Measurement

olmalıdır.

---

26. Cache Invalidation

Cache uygulanırsa recipe veya ingredient değişikliklerinde eski cache invalidation problemi ortaya çıkar.

Örneğin admin:

Recipe changed

sonrasında eski recommendation cache kullanılmamalıdır.

Bu nedenle:

Recipe update
↓
Related cache invalidation

veya versioned cache yaklaşımı kullanılabilir.

---

27. Redis Kullanımı

MVP'de Redis zorunlu değildir.

İleride:

Flask
↓
Redis
↓
PostgreSQL

mimarisi kullanılabilir.

Redis kullanım alanları:

Recommendation cache
Ingredient search cache
Rate limiting

Temporary data
Session-related data

olabilir.

Ancak Redis yalnızca gerçekten performans ihtiyacı ortaya çıktığında eklenmelidir.

---

28. Database Connection Pooling

Flask production ortamında PostgreSQL bağlantıları için connection pooling kullanılmalıdır.

SQLAlchemy connection pool:

Application
↓
Connection Pool
↓
PostgreSQL

şeklinde çalışır.

Amaç:

Her request → New DB connection

yerine mevcut bağlantıların verimli şekilde yeniden kullanılmasını sağlamaktır.

---

29. Connection Pool Configuration

Production'da aşağıdaki parametreler değerlendirilebilir:

pool_size
max_overflow
pool_timeout
pool_recycle

Değerler:

Server resources

Concurrent requests
PostgreSQL capacity

ile uyumlu belirlenmelidir.

Rastgele yüksek değerler verilmemelidir.

---

30. Database Query Optimization

Yavaş query tespit edildiğinde:

1. Query log
2. EXPLAIN
3. EXPLAIN ANALYZE
4. Index analysis
5. Query rewrite
6. ORM loading strategy

izlenmelidir.

Amaç sadece index eklemek değil, query'nin neden yavaş olduğunu anlamaktır.

---

31. N+1 Query Problemi

ORM kullanılan projelerde N+1 query problemine dikkat edilmelidir.

Örneğin:

1 query → recipes
100 queries → ingredients

gibi bir yapı performansı ciddi şekilde düşürebilir.

Bunun yerine uygun:

joined loading
selectin loading
explicit optimized query

yaklaşımları değerlendirilebilir.

---

32. Recipe Detail Query

Recipe detail ekranı için gerekli ilişkiler kontrollü şekilde yüklenmelidir.

Örneğin:

Recipe
└── RecipeIngredients
    └── Ingredient

ilişkisinin her biri için gereksiz ayrı query oluşturulmamalıdır.

Ancak bütün ilişkilerin tek query'de yüklenmesi de her zaman doğru değildir.

Loading strategy query kullanımına göre belirlenmelidir.

---

33. Transaction Performance

Transactionlar mümkün olduğunca:

short
atomic
predictable

olmalıdır.

Örneğin cart add:

Begin
↓
Find existing item
↓
Merge / Insert
↓
Commit

gereksiz uzun transaction içinde tutulmamalıdır.

---

34. Database Transaction Isolation

MVP'de PostgreSQL'in uygun default transaction isolation seviyeleri kullanılabilir.

Özel isolation seviyesi ancak gerçek concurrency problemi ortaya çıktığında değiştirilmelidir.

Özellikle cart merge işlemlerinde:

Concurrent add

durumu dikkate alınmalıdır.

Database unique constraint + transaction yaklaşımı duplicate kayıtları önlemede kullanılabilir.

---

35. Cart Merge Performance

Aynı kullanıcı:

Tomato + 2

ekledikten sonra:

Tomato + 3

eklerse:

Tomato = 5

olmalıdır.

Bunun için:

(user_id, ingredient_id, normalized unit)

üzerinden mevcut item hızlıca bulunmalıdır.

İlgili index bu işlemi desteklemelidir.

---

36. Favorites Performance

Favorite listesi:

GET /api/favorites

sırasında:

WHERE user_id = current_user_id

sorgusu kullanılır.

Bu nedenle:

favorites.user_id

index'i önemlidir.

Recipe bilgileri de gerekiyorsa N+1 query oluşmaması için uygun query strategy kullanılmalıdır.

---

37. API Response Compression

Production'da response compression değerlendirilebilir.

Özellikle JSON response'lar için:

gzip
brotli

gibi yöntemler kullanılabilir.

Ancak küçük response'larda compression overhead gereksiz olabilir.

Bu nedenle deployment seviyesinde ölçüm yapılarak etkinleştirilmelidir.

---

38. HTTP Keep-Alive

Mobil uygulama ile API arasındaki network bağlantılarında connection reuse desteklenmelidir.

Amaç:

Every request
↓
New TCP/TLS connection

yerine mümkün olduğunda mevcut bağlantının kullanılabilmesidir.

Bu durum latency'yi azaltabilir.

---

39. Timeout Strategy

Flutter ve backend tarafında timeout bulunmalıdır.

Örneğin:

Connection timeout
Receive timeout
Send timeout
Database timeout

tanımlanabilir.

Sonsuz bekleme:

No timeout

kullanılmamalıdır.

---

40. Retry Strategy

Her request otomatik olarak tekrar gönderilmemelidir.

Retry özellikle:

GET
idempotent operations
temporary network failures

için değerlendirilebilir.

Ancak:

POST /cart/items
POST /favorites

gibi işlemlerde kontrolsüz retry duplicate işlemlere yol açabilir.

Bu nedenle retry strategy endpoint bazlı tasarlanmalıdır.

---

41. Offline Considerations

MVP'de tam offline-first architecture zorunlu değildir.

Ancak Flutter:

No Internet

durumunu kullanıcıya anlaşılır şekilde göstermelidir.

Örneğin:

Internet connection unavailable.
Please try again.

Retry butonu sunulabilir.

İleride:

Offline cache
Queued actions
Sync

eklenebilir.

---

42. Local Cache

Flutter tarafında cache kullanılabilecek veriler:

Ingredient list
Recently loaded recipes
Images
User profile

olabilir.

Ancak authentication token gibi hassas veriler normal cache mekanizmalarında tutulmamalıdır.

---

43. Cache TTL

Cache uygulanırsa her veri için aynı süre kullanılmamalıdır.

Örnek:

Images → Long TTL
Ingredients → Medium TTL
Recipe detail → Medium TTL
User cart → Very short / server authoritative

Cart gibi değişken verilerde stale data problemi daha önemlidir.

---

44. Server as Source of Truth

Favorites ve cart için backend source of truth olarak kabul edilmelidir.

Flutter local cache:

UI performance

amacıyla kullanılabilir.

Ancak:

Server State
↓
Authoritative

olmalıdır.

Özellikle login sonrası senkronizasyon yapılmalıdır.

---

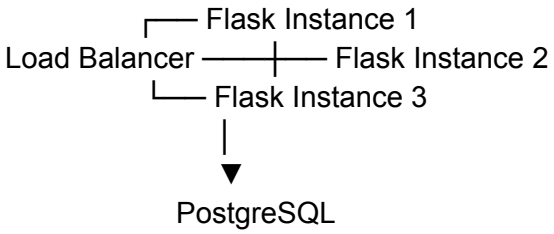
45. API Scalability

İlk aşamada:

1 Flask Application
1 PostgreSQL

yeterli olabilir.

Trafik arttığında:



şeklinde yatay ölçekleme yapılabilir.

---

46. Stateless Backend

Flask backend mümkün olduğunca stateless tutulmalıdır.

JWT yaklaşımı bu mimariyi destekler.

Örneğin:

Request 1 → Server A
Request 2 → Server B
Request 3 → Server C

aynı kullanıcı için çalışabilmelidir.

Sunucu lokal memory'sinde kritik session state tutulmamalıdır.

---

47. Horizontal Scaling

Trafik büyüdüğünde:

CPU ↑
Memory ↑
Request count ↑

olması durumunda yeni Flask instance'ları eklenebilir.

Ancak stateless architecture korunmalıdır.

Shared state gerekiyorsa:

PostgreSQL
Redis
Object Storage

gibi merkezi servisler kullanılabilir.

---

48. PostgreSQL Scalability

Database büyüdüğünde:

Index optimization
Query optimization
Connection pooling
Vacuum / analyze
Hardware scaling
Read replicas

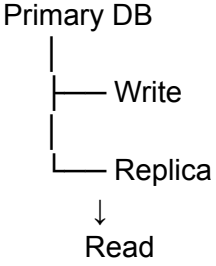
gibi yöntemler değerlendirilebilir.

MVP'de read replica zorunlu değildir.

---

49. Read Replica

İleri aşamada recipe ve ingredient gibi yoğun okuma işlemleri için:



mimarisi değerlendirilebilir.

Ancak replication lag nedeniyle bütün read işlemlerinin replica'ya yönlendirilmesi doğru değildir.

Örneğin:

Immediately after cart update

gibi işlemlerde güncel primary data gerekebilir.

---

50. Background Jobs

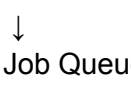
Uzun süren işlemler HTTP request içinde yapılmamalıdır.

İleride:

Recipe import
Image processing
Large dataset processing
Recommendation precomputation
Analytics processing

gibi işlemler background job olarak çalıştırılabilir.

Örneğin:



↓
Worker
↓
Task

mimarisi.

Celery/RQ gibi sistemler ihtiyaç ortaya çıktığında değerlendirilebilir.

---

51. AI Performance

Gelecekte AI/ML modeli sisteme eklendiğinde model inference süresi ayrıca ölçülmelidir.

Örneğin:

Request
↓
Ingredient Processing
↓
Model Inference
↓
Recommendation
↓
Response

Model inference:

CPU
GPU
Memory
Batching
Model size

gibi faktörlerden etkilenebilir.

MVP'deki deterministic matcher için ayrı bir ML inference altyapısı gerekmemektedir.

---

52. NLP / LLM Performance

Gelecekte doğal dil desteği eklenirse:

"Evde domates ve biraz peynir var."

gibi inputlardan ingredient extraction yapılabilir.

Ancak LLM çağrıları:

Latency ↑
Cost ↑
External dependency ↑

oluşturabilir.

Bu nedenle LLM:

User Input
↓
Ingredient Extraction
↓
Deterministic Recommendation Engine

şeklinde konumlandırılmalıdır.

LLM doğrudan bütün recipe selection sürecini yönetmemelidir.

---

53. Performance Monitoring

Production'da aşağıdaki metrikler izlenmelidir:

API request count
API latency
p50
p95
p99
HTTP 4xx
HTTP 5xx
Database latency
Database connection count
CPU
Memory
Disk
Recommendation latency

Cache hit rate

---

54. Endpoint Performance Metrics

Her önemli endpoint için:

Endpoint
Request Count
Average
p50
p95
p99
Error Rate

ölçülebilir.

Örneğin:

POST /api/recipes/recommend

Requests: 10,000
p50: 180 ms
p95: 620 ms
p99: 1.2 s

gibi.

Bu değerler örnektir; gerçek production ölçümü değildir.

---

55. Slow Query Monitoring

Database'te belirli bir threshold'un üzerinde çalışan query'ler izlenmelidir.

Örneğin:

Slow Query Threshold

belirlenebilir.

Yavaş query tespit edildiğinde:

SQL
↓
EXPLAIN ANALYZE
↓
Index / Join / Query structure
↓
Optimization
↓
Retest

uygulanmalıdır.

---

56. Load Testing

MVP release öncesinde temel load testing yapılmalıdır.

Test senaryoları:

Concurrent ingredient search
Concurrent recipe recommendation
Concurrent favorites
Concurrent cart operations
Concurrent authentication

Amaç:

At what traffic level does performance degrade?

sorusuna cevap bulmaktır.

---

57. Load Test Örneği

Örneğin:

10 concurrent users
50 concurrent users
100 concurrent users

gibi test seviyeleri uygulanabilir.

Her seviyede:

p95 latency
error rate
CPU
memory
database connections

ölçülmelidir.

Sayılar gerçek deployment kapasitesine göre belirlenmelidir.

---

58. Stress Testing

Stress test normal yükün üzerine çıkmayı amaçlar.

Örneğin:

Normal:
100 req/min

Stress:
500 req/min
1000 req/min

gibi seviyeler test edilebilir.

Amaç uygulamanın:

failure point
recovery behavior
resource exhaustion

davranışını gözlemlemektir.

---

59. Flutter Performance Testing

Flutter tarafında:

Frame rendering
Jank
Memory usage
Startup time
Image loading
List scrolling

test edilmelidir.

Özellikle:

Recipe Results
Cart
Favorites
Ingredient Search

ekranları test edilmelidir.

---

60. Memory Management

Flutter'da gereksiz büyük nesneler memory'de tutulmamalıdır.

Özellikle:

Large images
Large JSON
Huge lists

kontrol edilmelidir.

Backend'de de:

All recipes
All ingredients

gibi büyük datasetlerin gereksiz şekilde memory'ye alınmasından kaçınılmalıdır.

---

61. Database Memory

PostgreSQL performansı yalnızca CPU'ya bağlı değildir.

İzlenebilecek değerler:

Shared buffers
Cache hit ratio
Active connections
Disk I/O
Query duration

olabilir.

Production tuning deployment kaynaklarına göre yapılmalıdır.

---

62. Resource Limits

Backend için:

Maximum request body
Maximum page size
Maximum ingredient selection
Maximum cart quantity
Timeout

gibi limitler belirlenmelidir.

Amaç hem güvenlik hem performansı korumaktır.

---

63. Performance vs Correctness

Performans uğruna business logic bozulmamalıdır.

Örneğin recommendation cache kullanılırken:

Recipe changed

sonrasında eski recommendation döndürülmemelidir.

Benzer şekilde cart optimization:

Merge

işleminin veri doğruluğunu bozmamalıdır.

Öncelik:

Correctness
>
Security
>
Reliability
>
Performance

şeklinde değerlendirilmelidir.

---

64. Performance Anti-Patterns

Aşağıdaki uygulamalardan kaçınılacaktır:

❌ Her karakterde search request göndermek
❌ Tüm ingredientleri pagination olmadan çekmek
❌ Tüm tarifleri memory'ye almak
❌ N+1 query oluşturmak
❌ Her image'ı full resolution indirmek
❌ Her widget rebuild'inde ağır hesaplama yapmak
❌ Her request için yeni database connection oluşturmak
❌ Sonsuz timeout kullanmak
❌ Her POST request'e kontrolsüz retry uygulamak
❌ Gereksiz büyük JSON response döndürmek
❌ Ölçmeden cache eklemek
❌ Ölçmeden microservice architecture'a geçmek
❌ Performans için business rule'ları kaldırmak

---

65. Performance Development Workflow

Yeni bir özellik geliştirilirken:

Requirement
↓
Architecture
↓
Implementation
↓
Functional Test
↓
Performance Measurement
↓
Optimization if needed
↓
Regression Test

uygulanmalıdır.

---

66. Performance Debugging Workflow

Bir endpoint yavaşsa:

1. Problemi ölç
2. API latency kontrol et
3. Database latency kontrol et
4. Query sayısını kontrol et
5. Slow query tespit et
6. EXPLAIN ANALYZE çalıştır
7. Index kontrol et
8. Serialization kontrol et
9. Network payload kontrol et
10. Flutter rendering kontrol et

Rastgele optimizasyon yapılmamalıdır.

---

67. Performance Priority

P0

Pagination
Database indexes

Efficient recommendation query
N+1 prevention
Image caching
API timeout
Efficient Flutter lists
Search debounce
Connection pooling

P1

Response compression
Recommendation cache
Advanced query monitoring
Load testing
Performance dashboard

P2

Redis
Read replicas
Background workers
CDN
Advanced recommendation precomputation
ML inference optimization

---

68. MVP Performance Checklist

Release öncesi:

[ ] Ingredient search hızlı
[ ] Ingredient pagination mevcut
[ ] Recipe result pagination uygun
[ ] Recommendation query optimize edildi
[ ] Database indexes oluşturuldu
[ ] N+1 query kontrol edildi
[ ] Cart queryleri optimize edildi
[ ] Favorite queryleri optimize edildi
[ ] Image caching aktif
[ ] Search debounce aktif
[ ] API timeout mevcut
[ ] Connection pooling yapılandırıldı
[ ] Büyük response payloadları kontrol edildi

[ ] Flutter listeleri lazy rendering kullanıyor
[ ] Memory leak kontrol edildi
[ ] Temel load test yapıldı
[ ] p95 latency ölçüldü
[ ] 5xx oranı kontrol edildi

---

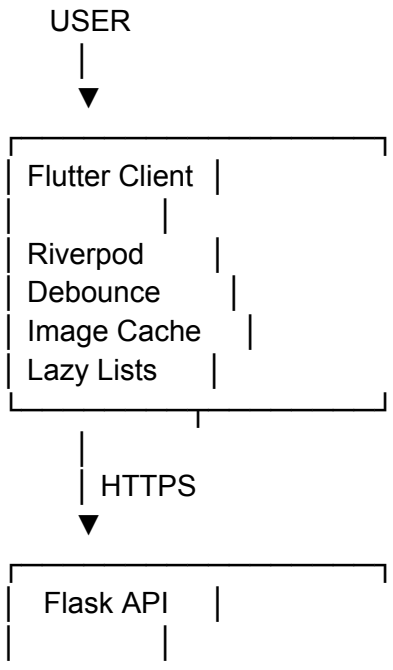
69. Definition of Performance Done

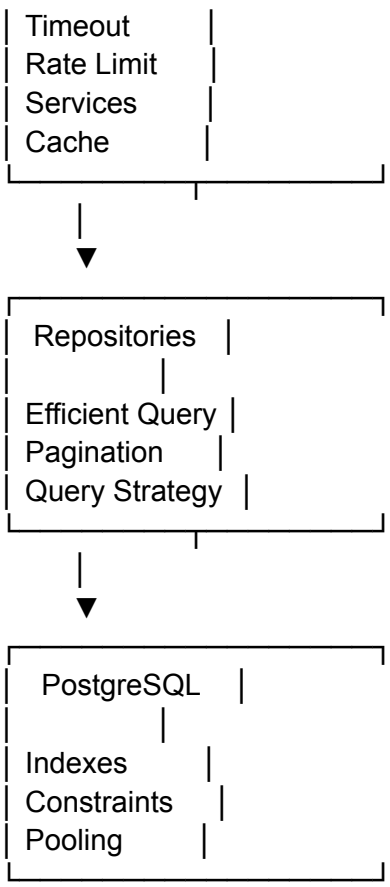
Bir özellik performance açısından tamamlanmış sayılması için:

[ ] Functional requirements tamamlandı
[ ] API response gereksiz veri içermiyor
[ ] Database queryleri kontrol edildi
[ ] Gerekli indexler mevcut
[ ] N+1 problemi kontrol edildi
[ ] Pagination değerlendirildi
[ ] Timeout belirlendi
[ ] Flutter rebuild davranışı kontrol edildi
[ ] Image/network kullanımı kontrol edildi
[ ] Performance testleri yapıldı
[ ] Regression testleri geçti

---

70. Nihai Performance Mimarisi




Timeout
Rate Limit
Services
Cache
Repositories
Efficient Query
Pagination
Query Strategy
PostgreSQL
Indexes
Constraints
Pooling

---

71. Sonuç

Smart Chef & Pantry AI için performans yaklaşımı, uygulamayı ilk günden gereksiz şekilde karmaşıklaştırmak yerine doğru mimari ve ölçüm temeline dayanmaktadır.

MVP'de temel öncelikler:

Efficient Queries
+
Pagination
+
Indexes
+
N+1 Prevention
+
Image Caching
+
Search Debounce
+
Timeout
+
Connection Pooling
+
Performance Monitoring

olacaktır.

Redis, CDN, read replica, background worker ve gelişmiş recommendation cache gibi teknolojiler MVP'nin zorunlu parçaları değildir. Bunlar gerçek trafik ve performans ölçümleri sonucunda ihtiyaç ortaya çıkarsa sisteme dahil edilecektir.

Böylece Smart Chef & Pantry AI:

Basit MVP
↓
Ölçülebilir Sistem
↓
Optimize Edilmiş Sistem
↓
Gerektiğinde Ölçeklenebilir Sistem

yaklaşımıyla geliştirilecektir.